

Sistema de Gerenciamento de Tarefas

Objetivo

Desenvolver um sistema de gerenciamento de tarefas utilizando **Angular** no frontend e **Java (Spring Framework e Hibernate)** no backend, com um banco de dados relacional em **SQL**. O sistema deve ser funcional, responsivo e apresentar boa organização de código.

Funcionalidades Requeridas

Frontend (Angular):

1. Interface para Tarefas:

- Criar um componente principal para **listar, criar, atualizar e excluir tarefas**.
- **Exibir tarefas em uma tabela com paginação:**
 - Colunas: título, descrição, status, data de criação e deadline.
- **Filtros:**
 - Filtro por **status** (PENDING, IN_PROGRESS, COMPLETED).
 - Ordenação por **deadline**.

2. Modal para Adicionar/Editar Tarefas:

- Criar um modal reutilizável.
- Validação amigável ao usuário (mensagens claras e feedbacks visuais).

3. Mensagens e Feedbacks:

- Notificações para ações bem-sucedidas (criação, edição, exclusão).
- Exibir mensagens de erro/validação em campos obrigatórios e no envio de dados.

4. Autenticação:

- Tela de login e registro.
- Feedbacks em caso de erro (e.g., credenciais inválidas).

Backend (Java - Spring Boot com Hibernate):

1. Autenticação:

- Registro de usuários (nome, e-mail, senha).
- Autenticação com geração de token **JWT**.

2. Endpoints de Tarefas:

- **GET /tasks**: Listar todas as tarefas com paginação e ordenação.
- **GET /tasks/{id}**: Buscar uma tarefa específica.
- **POST /tasks**: Criar uma nova tarefa.
- **PUT /tasks/{id}**: Atualizar uma tarefa existente.

- **DELETE /tasks/{id}**: Excluir uma tarefa.
- **GET /tasks/filter?status={status}**: Filtrar tarefas por status.
- 3. **Regras de Negócio (Camada de Serviço):**
 - Criar uma camada de serviço para encapsular lógica, como:
 - Validação de duplicidade no título.
 - Geração automática de **createdOn** ao criar uma tarefa.
 - Validação de permissões (e.g., usuário só pode acessar suas tarefas).
- 4. **Banco de Dados (Entidades):**

User

- **id**: Identificador único.
 - **username**: Nome do usuário.
 - **email**: E-mail do usuário.
 - **password**: Senha (armazenada de forma segura, e.g., hashed).
5. **Task**
- **id**: Identificador único.
 - **title**: Título da tarefa.
 - **description**: Descrição da tarefa.
 - **status**: Enum (PENDING, IN_PROGRESS, COMPLETED).
 - **createdOn**: Data de criação.
 - **deadline**: Data limite para conclusão.
 - **assignedTo**: Relacionamento com **User** (usuário responsável).

Entregáveis

Frontend:

1. Repositório no GitHub com:
 - Código organizado por módulos e componentes.
 - Interface funcional com mensagens de erro e validação.
2. Funcionalidades:
 - Listagem com paginação.
 - Filtros e ordenação por deadline e status.
 - Modal para criar/editar tarefas.
 - Mensagens amigáveis para feedbacks.

Backend:

1. Repositório no GitHub com:
 - API RESTful funcional e bem documentada.
 - Endpoints de autenticação e gerenciamento de tarefas.
 - Scripts SQL para criação do banco.
2. Funcionalidades:
 - Autenticação com JWT.
 - Operações CRUD nas tarefas.
 - Filtros e paginação.
3. Validações:
 - Campos obrigatórios nas entidades.
 - Deadline não pode ser uma data passada.
 - Usuários só podem acessar suas próprias tarefas.

Critérios de Avaliação

- **Frontend:**
 - Boa organização do projeto e modularidade.
 - Interface funcional, intuitiva e responsiva.
 - Uso correto de Angular Material ou outro framework de UI.
- **Backend:**
 - Uso adequado de Spring Boot e Hibernate.
 - Implementação correta da lógica de negócio na camada de serviço.
 - Segurança básica na API (JWT, validação de dados).
- **Documentação:**
 - Explicação dos endpoints da API e exemplos de uso.